
MICHWAR

Design, Migration, and Deployment of a Self-Hosted Backend for a Ride-Hailing Platform

An IMRAD Technical Report

ALLALI MOHAMMED ABDESSETTAR

June 2026

Abstract

This report documents the design, backend migration, build, and public deployment of **MICHWAR**, a unified passenger/driver ride-hailing application built with Flutter and targeting the Algerian market. The project originally relied on Google Firebase (Firestore, Authentication, Cloud Storage, and Cloud Functions) for its backend. To remove vendor lock-in, avoid recurring Firebase billing, and gain full control over the data model and business logic, the entire backend was re-implemented on **PocketBase**, a single-binary, self-hostable backend that combines an embedded SQLite database, a REST/realtime API, file storage, and a JavaScript hook runtime (JSVM) for server-side business logic. This report follows the IMRAD structure (Introduction, Methods, Results, and Discussion). The *Methods* section details the system architecture, the nine-collection PocketBase schema, the Firebase-to-PocketBase migration mapping, the dynamic commission and loyalty pricing engine, the geohash-based driver-matching algorithm, the Riverpod-based client state management, the Android release-build process, and the containerized deployment to Fly.io. The *Results* section reports the resulting codebase composition, a successful 54 MB release APK build, a live publicly reachable PocketBase instance at <https://michwar-pb.fly.dev>, and a worked example of the fare/commission computation. It also documents, with a timeline, the concrete build and deployment issues encountered (missing Android platform project, Windows shell quoting pitfalls, a PocketBase SDK API-version mismatch, and a PocketBase collection-name collision that crashed the deployed container) and how each was diagnosed and resolved. The *Discussion* reflects on the trade-offs of self-hosting versus a managed BaaS, the robustness benefits of enforcing all financial mutations through transactional hook routes, and lessons learned about cross-platform tooling. The project demonstrates a complete, reproducible path from a BaaS-dependent mobile application to an independently deployable, free-tier-hosted full-stack system.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Scope and Report Structure	4

2	Methods	4
2.1	Overall System Architecture	4
2.2	Technology Stack	4
2.3	Data Model and Schema Design	4
2.4	Migration Methodology: Firebase to PocketBase	5
2.5	Backend Business Logic (<code>pb_hooks</code>)	8
2.5.1	Shared Constants and the Pricing Engine	8
2.5.2	Geohash-Based Driver Matching	10
2.5.3	The Ride Lifecycle State Machine	11
2.6	Client Architecture: State Management and Realtime Data	13
2.7	Build Process: Producing a Release APK	15
2.8	Deployment Methodology: Containerization and Fly.io	16
3	Results	17
3.1	Codebase Composition	17
3.2	Build Results	17
3.3	Deployment Outcome	19
3.4	Worked Example: Fare and Commission Computation	19
3.5	Issues Encountered and Resolutions	19
4	Discussion	22
4.1	Architectural Trade-offs: Self-Hosting versus a Managed BaaS	22
4.2	Robustness of the Transactional Hook Pattern	22
4.3	Lessons Learned	22
4.4	Limitations	23
4.5	Future Work	23
5	Conclusion	23

1 Introduction

1.1 Background and Motivation

Ride-hailing platforms are inherently full-stack systems: a mobile client handles booking, navigation, and payment interactions, while a backend must provide authentication, a strongly consistent data store for ride state, real-time location updates, and server-enforced business rules for pricing, commissions, and driver payouts. **MICHWAR** is such a platform, designed for the Algerian market (currency: Algerian Dinar, DZD), supporting both passenger and driver roles within a single Flutter codebase.

The project was initially prototyped against **Google Firebase**: Firestore for data, Firebase Authentication (phone OTP) for sign-in, Cloud Storage for driver documents, and Cloud Functions for the authoritative, server-side business logic (ride matching, fare computation, wallet ledgering, ratings, and loyalty points). While Firebase enabled rapid prototyping, it introduces three durable constraints for a project intended for independent, long-term operation:

1. **Vendor lock-in** — the data model, security rules, and serverless functions are expressed in Firebase-specific APIs and are not portable without substantial rewriting.
2. **Recurring cost at scale** — Firestore reads/writes, Cloud Functions invocations, and egress are billed per-usage, with costs that grow with the number of active rides and realtime listeners.
3. **Operational opacity** — the operator has limited visibility into, and no direct access to, the underlying data store.

1.2 Problem Statement

The core engineering problem addressed by this project is: *re-implement the full MICHWAR backend — authentication, data model, business logic, file storage, and realtime updates — on a self-hosted, open-source backend, without changing the user-facing behaviour of the Flutter application, and then make the resulting system trivially deployable so that any user can download a release APK and have it work immediately, from anywhere, with no manual server configuration.*

1.3 Objectives

The work reported here covers the following concrete objectives:

1. Select and integrate a self-hosted backend (**PocketBase**) that can replace Firestore, Firebase Auth, Cloud Storage, and Cloud Functions with minimal client-side disruption.
2. Design a normalized PocketBase collection schema that mirrors the original Firestore document structure while taking advantage of PocketBase’s relational fields, file fields, and API rules.
3. Port all server-authoritative business logic (driver matching, dynamic commission tiers, surcharge, wallet ledger, loyalty points, ratings, SOS, live-trip sharing, demand heatmap) from Firebase Cloud Functions to PocketBase JSVM hooks (`pb_hooks/*.pb.js`).
4. Update the Flutter client (`lib/`) to use the `pocketbase` Dart SDK in place of the Firebase SDKs, including Riverpod providers, realtime subscriptions, and authentication state.
5. Produce a signed/release Android APK (`flutter build apk -release`) and resolve the platform-, tooling-, and dependency-version issues encountered along the way.
6. Deploy the PocketBase backend to a publicly reachable, free-tier cloud host (**Fly.io**) using a Docker container and a persistent volume, so that the distributed APK works for any user on any network.

1.4 Scope and Report Structure

Section 2 (*Methods*) describes the resulting system architecture, data model, business-logic implementation, client-side state management, and the build/deployment pipeline, with representative code listings. Section 3 (*Results*) reports the quantitative and qualitative outcomes: codebase composition, build artifacts, the live deployment, a worked fare-computation example, and a chronological account of the issues encountered and how they were resolved. Section 4 (*Discussion*) reflects on the architectural trade-offs, lessons learned, limitations, and possible future work. Section 5 concludes the report.

2 Methods

2.1 Overall System Architecture

MICHWAR follows a **feature-first** Flutter architecture on the client side and a **single-binary** backend on the server side. Each feature folder under `lib/features/` (`authentication`, `passenger`, `driver`, `ride_engine`, `shared`) owns its own screens, Riverpod providers, and repository services, while `lib/core/` holds everything shared across features: data models, app-wide constants, theming, `go_router`-based navigation, and cross-cutting services (PocketBase client, authentication, location, connectivity, notifications). On the server side, a single PocketBase binary serves the REST/realtime API, persists data in an embedded SQLite database, stores uploaded files, and executes the custom JavaScript business logic in `pb_hooks/`. Figure 1 summarizes how the client feature modules map onto PocketBase collections and hook routes.

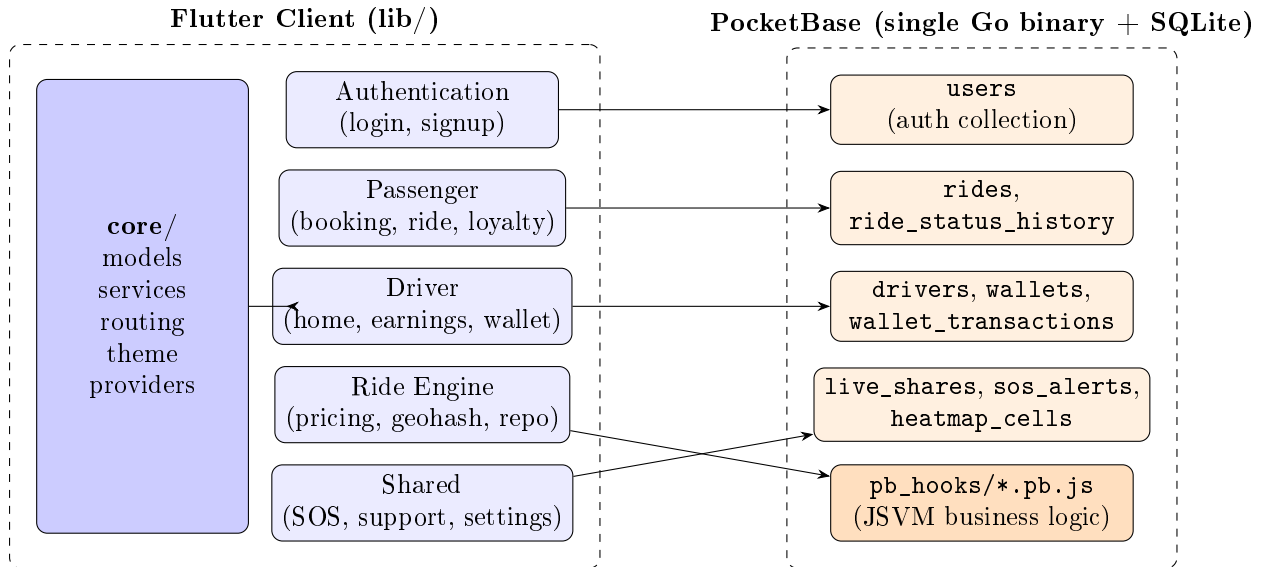


Figure 1: High-level architecture: feature-first Flutter modules communicate with PocketBase collections directly (CRUD + realtime subscriptions) for non-sensitive reads/writes, and through custom `pb_hooks` routes for every authoritative, financially sensitive mutation.

2.2 Technology Stack

2.3 Data Model and Schema Design

PocketBase organizes data into *collections*, each backed by a SQLite table with a JSON-schema-like field definition, optional relation fields, file fields, and per-operation API rules (`list`, `view`, `create`, `update`, `delete`) evaluated server-side. The MICHWAR schema consists of nine collections, defined declaratively in `pocketbase/pb_migrations/*.js` and applied automatically

Table 1: Technology stack summary

Layer	Technology
Client framework	Flutter 3.43 (beta channel), Dart SDK $\geq 3.3.0$
State management	<code>flutter_riverpod</code> 2.6.1 (StreamProvider-based realtime state)
Navigation	<code>go_router</code> 14.3.0
Backend	PocketBase 0.23.x (server), <code>pocketbase</code> Dart SDK 0.24.0 (client)
Database	Embedded SQLite (managed entirely by PocketBase)
Realtime transport	Server-Sent Events (SSE) via PocketBase <code>subscribe()</code>
Local persistence	<code>shared_preferences</code> (auth token), <code>hive/hive_flutter</code> (offline cache)
Maps & location	<code>google_maps_flutter</code> , <code>geolocator</code> , <code>geocoding</code> , <code>flutter_polyline_points</code>
Backend hosting	Docker (Alpine multi-stage build), Fly.io (Firecracker micro-VMs)
Build target	Android release APK (<code>flutter build apk -release</code>)

the first time the server starts. Table 2 summarizes the collections and their write ownership: *Client* fields are written directly by the Flutter app subject to API rules; *Hook* fields are written exclusively by `pb_hooks` routes running with full (`$app`) privileges, with `update/delete` API rules set to `null` so that ordinary authenticated requests cannot bypass the business logic.

2.4 Migration Methodology: Firebase to PocketBase

The migration was carried out as a systematic, document-by-document mapping from the Firestore data model and Cloud Functions to the PocketBase equivalent, recorded in `pocketbase/MIGRATION_PLAN.m` and summarized in Table 3.

A representative migration script is shown in Listing 1, which defines the `users` auth collection. Each migration file exports an `up` function (applied on first boot, or when the migration has not yet run) and a `down` function (used for rollback). One subtlety — discovered during deployment and discussed in §3.5 — is that PocketBase ≥ 0.23 **auto-creates a default `users` auth collection** on a fresh database, so the migration must first remove that bootstrap collection before creating the application’s own `users` collection with its custom fields and API rules.

```

1  /// <reference path="../../pb_data/types.d.ts" />
2  migrate((app) => {
3    // PocketBase >= 0.23 bootstraps a default "users" auth collection on a
4    // fresh instance, which collides with the one this migration creates
5    // ("Collection name must be unique (case insensitive)"). Since this is
6    // the very first migration (runs before anything references "users"),
7    // it's safe to drop the bootstrap collection and replace it with ours.
8    try {
9      const existing = app.findCollectionByNameOrId("users");
10     if (existing) {
11       app.delete(existing);
12     }
13   } catch (e) {
14     // no existing "users" collection -- nothing to remove
15   }
16
17   const collection = new Collection({
18     "name": "users",

```

Table 2: PocketBase collections and their roles

Collection	Purpose / key fields
users (auth)	Email/password auth collection. Profile fields: <code>fullName</code> , <code>phoneNumber</code> , <code>role</code> (<code>passenger/driver/admin</code>), <code>avatar</code> , <code>loyaltyPoints</code> , <code>loyaltyTier</code> , <code>totalRidesCompleted</code> , <code>savedPlaces</code> , <code>sosContacts</code> , <code>fcmTokens</code> .
drivers	1:1 relation to users . Vehicle info, verification status/-documents, commission tier/rate, ratings, online/on-trip flags, live location + geohash, wallet balance.
rides	Core ride record: passenger/driver relations, status (state machine, see §2.5.3), pickup/dropoff JSON, pickupGeohash , candidateDriverIds , fare JSON (authoritative), rating , live-share token, full timestamp trail.
ride_status_history	Append-only audit log: one row per status transition, with the acting user.
wallets	1:1 relation to driver; current balance and low-balance flag.
wallet_transactions	Immutable ledger of every wallet movement (ride earnings, top-ups), with the full fare breakdown.
live_shares	Short-lived (6h) publicly readable tokens for live trip tracking.
sos_alerts	Emergency alerts with ride context and reporter location/contacts.
heatmap_cells	Aggregated demand counters keyed by a coarse geohash, incremented atomically.

Table 3: Firebase → PocketBase mapping (excerpt)

Firestore	Firebase construct	PocketBase equivalent	Notes
	Firebase Auth (phone OTP)	<code>users</code> auth collection (email + password)	<code>phoneNumber</code> becomes a plain profile field
	<code>users/{uid}</code> (Firestore)	<code>users</code> record	Same id reused as record id
	<code>drivers/{uid}</code>	<code>drivers</code> (relation <code>user</code>)	1:1 relation instead of shared document id
	<code>rides/{id}</code>	<code>rides</code>	<code>passengerId/driverId</code> strings become relation fields
	<code>rides/{id}/status_history</code>	<code>ride_status_history</code>	relation <code>ride</code> + <code>actor</code>
	<code>wallets/{driverId}</code>	<code>wallets</code> (relation <code>driver</code>)	1:1 relation
	Cloud Storage driver docs	<code>drivers.documents</code> (multi-file field)	+ <code>documentsMeta</code> JSON for review status
	<code>onRideRequestCreated</code> trigger	<code>onRecordAfterCreateSuccessful</code> hook on <code>rides</code>	Cash driver matching
	<code>acceptRide</code> callable	POST <code>/api/michwar/rides/:id/accept</code>	
	<code>updateRideStatus</code> callable	POST <code>/api/michwar/rides/:id/status</code>	
	<code>completeRide</code> callable	POST <code>/api/michwar/rides/:id/complete</code>	Authoritative fare split
	<code>submitRating</code> callable	POST <code>/api/michwar/rides/:id/rate</code>	
	<code>generateLiveShareLink</code> callable	POST <code>/api/michwar/rides/:id/share</code>	

```

19     "type": "auth",
20     "fields": [
21       { "name": "fullName", "type": "text", "required": true, "max": 120 },
22       { "name": "phoneNumber", "type": "text", "required": true, "max": 20 },
23       { "name": "role", "type": "select", "required": true, "maxSelect": 1,
24         "values": ["passenger", "driver", "admin"] },
25       { "name": "roleSelected", "type": "bool" },
26       { "name": "avatar", "type": "file", "maxSelect": 1, "maxSize": 5242880,
27         "mimeType": ["image/png", "image/jpeg", "image/webp"] },
28       { "name": "loyaltyPoints", "type": "number" },
29       { "name": "loyaltyTier", "type": "select", "maxSelect": 1,
30         "values": ["standard", "eco", "premium"] },
31       { "name": "totalRidesCompleted", "type": "number" },
32       { "name": "savedPlaces", "type": "json", "maxSize": 200000 },
33       { "name": "sosContacts", "type": "json", "maxSize": 200000 },
34       { "name": "fcmTokens", "type": "json", "maxSize": 50000 }
35     ],
36     "passwordAuth": { "enabled": true, "identityFields": ["email"] },
37     "listRule": null,
38     "viewRule": "id = @request.auth.id || @request.auth.role = 'admin'",
39     "createRule": "",
40     "updateRule": "id = @request.auth.id",
41     "deleteRule": null
42   });
43
44   return app.save(collection);
45 }, (app) => {
46   const collection = app.findCollectionByNameOrId("users");
47   return app.delete(collection);
48 });

```

Listing 1: `pb_migrations/1700000001_users.js` — defines the `users` auth collection (with the bootstrap-collection guard added during deployment debugging).

2.5 Backend Business Logic (pb_hooks)

All authoritative, money-affecting, or matching-related logic lives in `pocketbase/pb_hooks/*.pb.js`, loaded automatically by `pocketbase serve` into a single shared JavaScript runtime (files are loaded in filename order, hence the numeric prefixes). This section describes the three most important hook subsystems: shared constants and pricing (§2.5.1), geohash-based driver matching (§2.5.2), and the ride lifecycle state machine (§2.5.3).

2.5.1 Shared Constants and the Pricing Engine

`00_constants.pb.js` defines a single global object, `MICHWAR`, holding every business constant (commission rates, surcharge bounds, wallet thresholds, loyalty thresholds, fare formula coefficients, geohash precision, etc.). This object mirrors — and must stay in sync with — `lib/core/constants/app_constants.dart` on the client, which uses the same constants *only* to show fare *estimates* before a ride is requested.

`02_pricing.pb.js` (Listing 2) implements the actual billing formulas. `mwComputeBaseFare` computes the base fare from the *actual* GPS-tracked distance and duration of a completed trip, applies a ride-tier multiplier (`standard` $\times 1.0$, `eco` $\times 0.92$, `premium` $\times 1.35$), enforces a minimum fare, and rounds to the nearest 5 DZD. `mwComputeSurcharge` derives a small (1–4 DZD) platform surcharge deterministically from the ride id, so repeated calls for the same ride are stable but different rides vary. `mwComputeFareBreakdown` then splits the base fare between the driver and the company according to the driver’s current **commission tier**.

```

1 function mwRoundToNearest(value, nearest) {
2   return Math.round(value / nearest) * nearest;

```

```

3 }
4
5 /** Computes the BASE FARE from actual trip distance/duration. */
6 function mwComputeBaseFare(distanceKm, durationMin, rideTier) {
7   var multiplier = MICHWAR.RIDE_TIER_MULTIPLIERS[rideTier];
8   if (multiplier == null) multiplier = 1.0;
9
10  var raw =
11    MICHWAR.BASE_FARE_FLAG_DZD +
12    distanceKm * MICHWAR.FARE_PER_KM_DZD +
13    durationMin * MICHWAR.FARE_PER_MINUTE_DZD;
14
15  var tiered = raw * multiplier;
16
17  return tiered < MICHWAR.MINIMUM_FARE_DZD
18    ? MICHWAR.MINIMUM_FARE_DZD
19    : mwRoundToNearest(tiered, 5);
20 }
21
22 /** Deterministic-but-varying platform surcharge in [MIN, MAX] DZD. */
23 function mwComputeSurcharge(rideId) {
24   var hash = 0;
25   for (var i = 0; i < rideId.length; i++) {
26     hash = (hash * 31 + rideId.charCodeAt(i)) >>> 0;
27   }
28   var range = MICHWAR.SURCHARGE_MAX_DZD - MICHWAR.SURCHARGE_MIN_DZD + 1;
29   return MICHWAR.SURCHARGE_MIN_DZD + (hash % range);
30 }
31
32 /** Splits the base fare between driver and company, adds the surcharge. */
33 function mwComputeFareBreakdown(baseFare, surchargeDzd, commissionRate) {
34   var commissionAmount = mwRoundToNearest(baseFare * commissionRate, 1);
35   var driverPayout = baseFare - commissionAmount;
36   var companyRevenue = commissionAmount + surchargeDzd;
37   var totalFare = baseFare + surchargeDzd;
38
39   return {
40     baseFare: baseFare, surchargeDzd: surchargeDzd, totalFare: totalFare,
41     commissionRate: commissionRate, commissionAmount: commissionAmount,
42     driverPayout: driverPayout, companyRevenue: companyRevenue,
43   };
44 }
45
46 /** Resolves a driver's commission tier from lifetime stats (one-way). */
47 function mwResolveCommissionTier(ridesCompleted, ratingAverage, currentTier) {
48   var alreadyElite = currentTier === "tier2";
49   var qualifiesForElite =
50     ridesCompleted >= MICHWAR.ELITE_MIN_COMPLETED_RIDES &&
51     ratingAverage >= MICHWAR.ELITE_MIN_AVERAGE_RATING;
52
53   if (alreadyElite || qualifiesForElite) {
54     return { tier: "tier2", commissionRate: MICHWAR.TIER2_COMMISSION_RATE,
55             driverShareRate: MICHWAR.TIER2_DRIVER_RATE };
56   }
57   return { tier: "tier1", commissionRate: MICHWAR.TIER1_COMMISSION_RATE,
58           driverShareRate: MICHWAR.TIER1_DRIVER_RATE };
59 }

```

Listing 2: `pb_hooks/02_pricing.pb.js` — the sole source of billed fare, commission, and loyalty numbers (abridged).

Two driver commission tiers exist: **Tier 1 (Base)**, where the company retains 15% of the base fare and the driver keeps 85%, and **Tier 2 (Elite)**, where the company retains only 7% and

the driver keeps 93%. Promotion to Elite requires at least 100 completed rides *and* an average rating of at least 4.0, and — once granted — is never revoked even if the average later dips (a one-way ratchet, implemented by the `alreadyElite` check). Figure 2 visualizes this split. The platform surcharge (1–4 DZD) is 100% company revenue and is *not* part of the commission base.

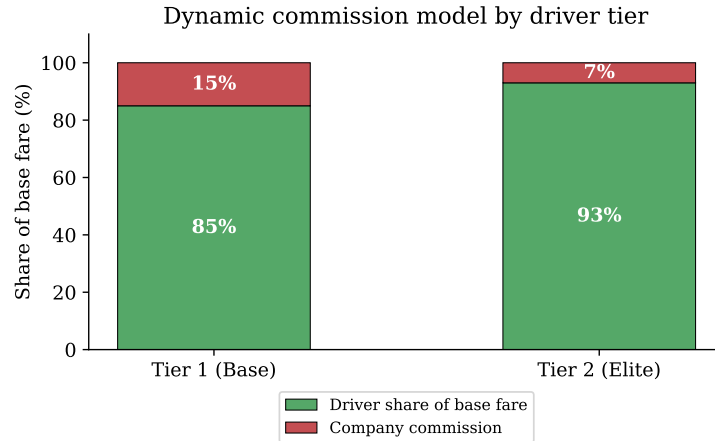


Figure 2: Dynamic commission model: the driver’s share of the base fare increases from 85% to 93% upon promotion to the Elite (Tier 2) tier, which requires ≥ 100 completed rides and an average rating ≥ 4.0 .

MICHWAR Points (the loyalty currency) are awarded on every completed ride as $\lfloor \text{totalFare}/100 \rfloor \times 2$. A passenger reaches the *eco* tier (5% discount on estimates) at 50 points and the *premium* tier (access to premium-tier rides) at 200 points, via `mwResolveLoyaltyTier`.

2.5.2 Geohash-Based Driver Matching

MICHWAR has no native geospatial index (PocketBase/SQLite has no built-in geo type), so proximity matching is implemented with **geohashing**: each driver’s location is encoded into a base-32 geohash string (`locationGeohash`, precision 7, $\approx 153\text{m} \times 153\text{m}$ cells) and stored as an indexed text field. The *same* geohash algorithm is implemented twice — once in Dart (`lib/core/utis/geohash_util.dart`, used for client-side estimates and the driver’s periodic location pings) and once in JavaScript (`pb_hooks/01_geohash.pb.js`, used server-side for matching) — to guarantee that cell boundaries agree between client and server.

When a passenger creates a ride request (`rides` record with `status = "searching"`), an `onRecordAfterCreateSuccess` hook on `rides` runs `mwMatchDriversForRide`. This function:

1. Starts at `DEFAULT_DRIVER_SEARCH_RADIUS_KM` (3 km).
2. Computes the geohash precision whose cell size is $\geq$ the current radius, enumerates the neighboring cells covering a circle of that radius (`mwNeighborsForRadius`), and queries `drivers` where `isOnline = true`, `isOnTrip = false`, `verificationStatus = 'approved'`, `walletBalance` is above the low-balance threshold, and `locationGeohash` falls within the cell-prefix range.
3. Filters the resulting candidates by true Haversine distance (`mwDistanceKm`) $\leq$ radius.
4. If no candidates are found, **doubles the radius** and repeats, up to `MAX_DRIVER_SEARCH_RADIUS_KM` (10 km); if still empty, sets `status = "no_drivers_found"`.
5. Otherwise sorts candidates by distance (with a "Heading Home" bonus, described below) and writes up to `MAX_CANDIDATE_DRIVERS` (10) driver ids into `rides.candidateDriverIds`, which each driver app subscribes to via realtime.

The **"Heading Home"** feature lets a driver mark a destination they are driving toward at the end of their shift. If a candidate driver has `headingHomeEnabled` and the bearing from their cur-

rent position to their home destination (`mwBearingDeg`) is within `headingHomeBearingTolerance` (default 30°) of the bearing to the ride's drop-off, their *effective* distance is reduced by 1 km, making them more likely to be offered a "last ride home" that is convenient for them. Figure 3 illustrates the expanding-radius search.

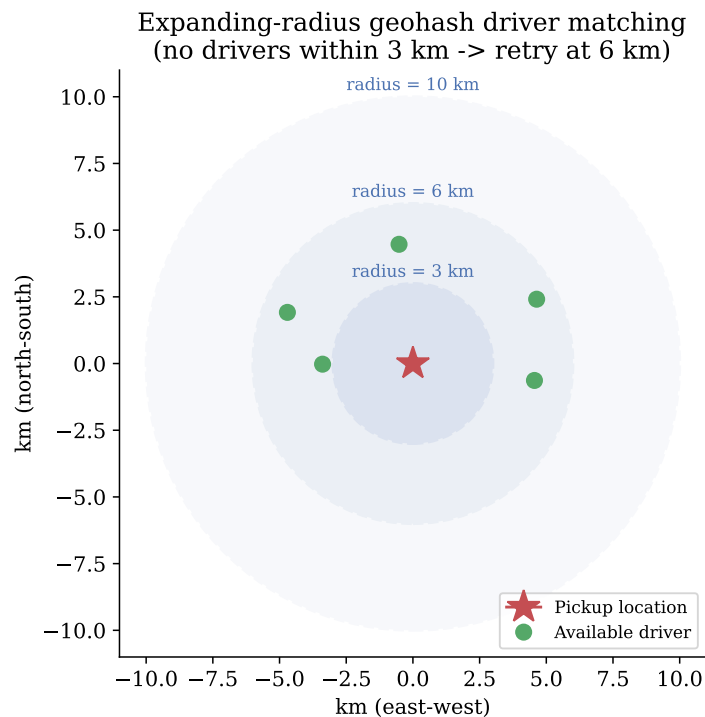


Figure 3: Expanding-radius geohash matching: the matcher first searches the 3 km cell neighborhood around the pickup point; if no eligible driver is found, it doubles the radius (6 km, then 10 km) until a candidate is found or the maximum radius is reached.

2.5.3 The Ride Lifecycle State Machine

`rides.status` follows a strict state machine, enforced entirely server-side. A passenger may only *create* a ride (which is forced to `status = "searching"`); every subsequent transition is performed by one of the `pb_hooks/12_rides.pb.js` routes running inside `$app.runInTransaction()`, which is PocketBase's equivalent of a Firestore transaction. Direct client `update/delete` on `rides` is disabled at the API-rule level (`updateRule: null`), and an `onRecordUpdateRequest` hook additionally reverts any of the protected fields (`status`, `driver`, `fare`, timestamps, etc.) listed in `MW_RIDE_PROTECTED_FIELDS` if a non-admin somehow submits an update. Figure 4 shows the resulting state machine.

Listing 3 shows the `/accept` route: it atomically checks that the ride is still `searching` and unclaimed, that the requesting driver is not already on a trip and has sufficient wallet balance, then assigns the driver, flips the ride to `accepted`, marks the driver `isOnTrip = true`, and appends a `ride_status_history` row — all inside one transaction so that two drivers can never simultaneously "win" the same ride.

```

1 routerAdd("POST", "/api/michwar/rides/{id}/accept", (e) => {
2   if (!e.auth) throw new UnauthorizedError("You must be signed in.");
3
4   var rideId = e.request.pathValue("id");
5
6   $app.runInTransaction((txApp) => {
7     var ride = txApp.findRecordById("rides", rideId);

```

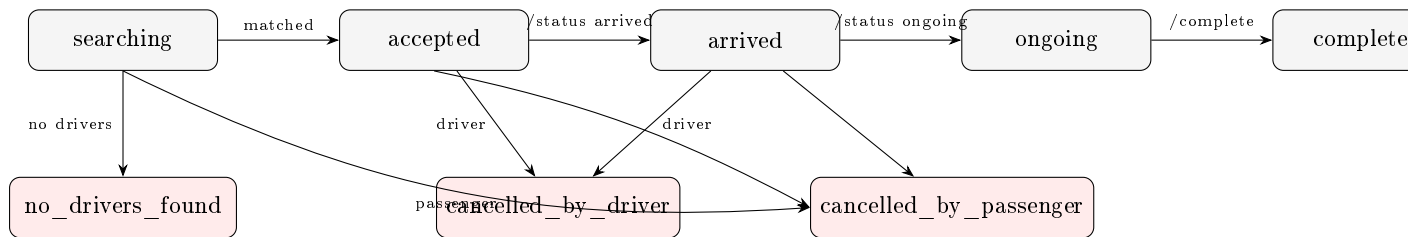


Figure 4: Ride lifecycle state machine implemented in `pb_hooks/12_rides.pb.js`. All transitions other than the initial `create` (passenger → `searching`) are performed inside a server-side transaction by one of the `/api/michwar/rides/:id/{accept,status,complete}` routes.

```

8   var driver = txApp.findFirstRecordByFilter(
9     "drivers", "user = {:uid}", { uid: e.auth.id });
10
11   if (ride.get("status") !== "searching" || ride.get("driver")) {
12     throw new BadRequestError(
13       "This ride has already been accepted by another driver.");
14   }
15   if (driver.get("isOnTrip")) {
16     throw new BadRequestError("You already have an active trip.");
17   }
18   if ((driver.get("walletBalance") || 0)
19     <= MICHWAR.WALLET_LOW_BALANCE_THRESHOLD_DZD) {
20     throw new BadRequestError(
21       "Your wallet balance is too low to accept new rides.");
22   }
23
24   ride.set("driver", e.auth.id);
25   ride.set("status", "accepted");
26   ride.set("acceptedAt", new Date().toISOString());
27   txApp.save(ride);
28
29   driver.set("isOnTrip", true);
30   txApp.save(driver);
31
32   var history = new Record(txApp.findCollectionByNameOrId("
33     ride_status_history"));
34   history.set("ride", rideId);
35   history.set("status", "accepted");
36   history.set("actor", e.auth.id);
37   txApp.save(history);
38 }
39
40 return e.json(200, { ok: true });
}, $apis.requireAuth());

```

Listing 3: `pb_hooks/12_rides.pb.js` — POST `/api/michwar/rides/:id/accept` (race-safe ride acceptance).

The `/complete` route (the most complex in the system, ≈ 130 lines) is invoked by the driver when a trip ends, supplying the GPS-tracked `actualDistanceKm` and `actualDurationMin`. Within a single transaction it: (1) computes the fare breakdown via `02_pricing.pb.js`; (2) debits the driver's wallet by the company's revenue share and writes the new balance to both `drivers` and `wallets`; (3) appends an immutable `wallet_transactions` ledger row containing the full breakdown; (4) increments the driver's `ridesCompleted` and re-evaluates their commission tier; (5) credits the passenger's loyalty points and re-evaluates their loyalty tier; and (6) writes the authoritative `fare` object onto the `rides` record and marks it `completed`. Because every one of these writes happens inside `$app.runInTransaction()`, a crash or error at any point rolls

back the entire set — there is no possibility of, for example, debiting a driver’s wallet without recording the corresponding ledger entry.

2.6 Client Architecture: State Management and Realtime Data

The Flutter client uses **Riverpod** (`flutter_riverpod`) for dependency injection and reactive state. Listing 4 shows an excerpt of `lib/core/providers/app_providers.dart`: the `pocketbaseProvider` exposes the shared `PocketBase` client (the instance is created once in `main.dart` via `PocketBaseService.getInstance()` and provided as an override — see Listing 5); `authStateProvider` is a `StreamProvider<String?>` derived from `AuthService.authStateChanges()`; and `watchRelatedRecord` is a generic helper that subscribes to the single record in a collection whose relation field matches the current user (used for both `driverProfileProvider` and `walletProvider`). Each subscription immediately emits the current value via `getFirstListItem`, then again on every realtime `create/update/delete` event delivered over the `PocketBase` SSE stream — functionally equivalent to a `Firestore DocumentReference.snapshots()` listener, but backed by `PocketBase`’s realtime API.

```

1  final pocketbaseProvider = Provider<PocketBase>((ref) {
2    throw UnimplementedError(
3      'pocketbaseProvider must be overridden in main.dart with the '
4      'PocketBaseService-initialized client.');
```

```

5  });
6
7  final authStateProvider = StreamProvider<String?>((ref) {
8    return ref.watch(authServiceProvider).authStateChanges();
9  });
10
11 final driverProfileProvider = StreamProvider<DriverModel?>((ref) {
12   final uid = ref.watch(authStateProvider).valueOrNull;
13   if (uid == null) return Stream.value(null);
14   final pb = ref.watch(pocketbaseProvider);
15   return watchRelatedRecord(
16     pb: pb, collection: PbCollections.drivers, relationField: 'user',
17     relationValue: uid, fromMap: DriverModel.fromMap,
18   );
19 });
20
21 Stream<T?> watchRelatedRecord<T>({
22   required PocketBase pb,
23   required String collection,
24   required String relationField,
25   required String relationValue,
26   required T Function(String id, Map<String, dynamic> map) fromMap,
27 }) {
28   final controller = StreamController<T?>();
29   UnsubscribeFunc? unsubscribe;
30
31   Future<void> start() async {
32     try {
33       final record = await pb.collection(collection).getFirstListItem(
34         pb.filter('$relationField = {value}', {'value': relationValue}));
35       controller.add(fromMap(record.id, record.toJson()));
36
37       unsubscribe = await pb.collection(collection).subscribe(record.id, (e) {
38         if (e.action == 'delete') {
39           controller.add(null);
40         } else if (e.record != null) {
41           controller.add(fromMap(e.record!.id, e.record!.toJson()));
42         }
43       });
44     } catch (_) {
```

```

45     controller.add(null);
46   }
47 }
48
49 controller.onListen = () {
50   start().catchError((Object error, StackTrace stack) {
51     controller.addError(error, stack);
52   });
53 };
54 controller.onCancel = () async {
55   await unsubscribe?.call();
56   await controller.close();
57 };
58
59 return controller.stream;
60 }

```

Listing 4: `lib/core/providers/app_providers.dart` — generic realtime "watch the record related to me" provider helper (abridged).

Listing 5 shows `pocketbase_service.dart`: it resolves the backend base URL via `pocketbaseBaseUrl()`, which defaults to `http://10.0.2.2:8090` on the Android emulator (the special address that routes to the host machine's localhost), `http://127.0.0.1: 8090` elsewhere, and is overridable at build time with `-dart-define=POCKETBASE_URL=...` — the mechanism used to point release builds at the public Fly.io deployment (§2.8). The auth session is persisted across app restarts using an `AsyncAuthStore` backed by `shared_preferences`, replacing Firebase Auth's automatic session persistence.

```

1 String pocketbaseBaseUrl() {
2   const override = String.fromEnvironment('POCKETBASE_URL');
3   if (override.isNotEmpty) return override;
4
5   if (kIsWeb) return 'http://127.0.0.1:8090';
6   if (Platform.isAndroid) return 'http://10.0.2.2:8090';
7   return 'http://127.0.0.1:8090';
8 }
9
10 class PocketBaseService {
11   PocketBaseService._(this.pb);
12   final PocketBase pb;
13   static PocketBaseService? _instance;
14
15   static Future<PocketBaseService> getInstance() async {
16     if (_instance != null) return _instance!;
17
18     final prefs = await SharedPreferences.getInstance();
19     final store = AsyncAuthStore(
20       save: (String data) async => prefs.setString('pb_auth', data),
21       initial: prefs.getString('pb_auth'),
22       clear: () async => prefs.remove('pb_auth'),
23     );
24
25     final pb = PocketBase(pocketbaseBaseUrl(), authStore: store);
26     _instance = PocketBaseService._(pb);
27     return _instance!;
28   }
29 }

```

Listing 5: `lib/core/services/pocketbase_service.dart` — PocketBase client singleton with persisted auth.

A notable implementation detail encountered during the build (detailed in §3.5) concerns the `pocketbase` Dart package's authentication-state stream. Version 0.24.0 exposes `AuthStore.onChange`

as a **Dart Stream<AuthStoreEvent>** (replacing the older callback-based API), and exposes the current user as `AuthStore.record` (an `AuthRecord?`) rather than the older `AuthStore.model`. Listing 6 shows the corrected `authStateChanges()` implementation, which wraps the SDK's stream in a `StreamController` so it can be combined with an immediate "current value" emission — mirroring the ergonomics of `FirebaseAuth.authStateChanges()`.

```

1  class AuthService {
2    AuthService(this._pb);
3    final PocketBase _pb;
4
5    bool get isSignedIn => _pb.authStore.isValid;
6    String? get currentUserId => _pb.authStore.record?.id;
7
8    /// Emits the current user id immediately, then again on every change;
9    /// 'null' means "signed out".
10   Stream<String?> authStateChanges() {
11     final controller = StreamController<String?>();
12     controller.add(_pb.authStore.record?.id);
13
14     final subscription = _pb.authStore.onChange.listen((event) {
15       controller.add(event.record?.id);
16     });
17
18     controller.onCancel = () {
19       subscription.cancel();
20       controller.close();
21     };
22
23     return controller.stream;
24   }
25
26   Future<RecordAuth> signUp({
27     required String email, required String password,
28     required String fullName, required String phoneNumber,
29   }) async {
30     await _pb.collection(PbCollections.users).create(body: {
31       'email': email, 'password': password, 'passwordConfirm': password,
32       'fullName': fullName, 'phoneNumber': phoneNumber, 'role': 'passenger',
33     });
34     return signIn(email: email, password: password);
35   }
36
37   Future<RecordAuth> signIn({required String email, required String password})
38   {
39     return _pb.collection(PbCollections.users).authWithPassword(email,
40       password);
41   }
42
43   Future<void> signOut() async => _pb.authStore.clear();
44 }

```

Listing 6: `lib/core/services/auth_service.dart` — authentication state stream over the PocketBase 0.24.0 `AuthStore.onChange` stream API.

2.7 Build Process: Producing a Release APK

Although the Flutter source tree (`lib/`) had been fully migrated, the project had originally been authored without ever running `flutter create`, so the Android (and iOS/web/desktop) platform projects did not exist. Running `flutter create .` from the project root scaffolds `android/`, `ios/`, `web/`, `windows/`, `linux/`, and `macos/` *without* touching the existing `lib/` source — a safe, additive operation. After `flutter pub get`, the release APK is produced with:

```

1 flutter build apk --release \
2   --dart-define=POCKETBASE_URL=https://michwar-pb.fly.dev

```

Listing 7: Release APK build command, with the PocketBase backend URL baked in as a compile-time constant.

`--dart-define` injects a compile-time environment constant that `pocketbaseBaseUrl()` (Listing 5) reads via `String.fromEnvironment('POCKETBASE_URL')` — this is how the same codebase produces a build that talks to 10.0.2.2:8090 (emulator), a LAN IP (local testing on a physical device), or the public Fly.io URL (distributable release), with no source changes.

2.8 Deployment Methodology: Containerization and Fly.io

For the distributed APK to "just work" for any user on any network, the PocketBase backend must be reachable over the public internet with HTTPS — a LAN IP (e.g. 192.168.1.50:8090) only works for devices on the same Wi-Fi as the development machine. The backend was therefore containerized and deployed to **Fly.io**, which offers a free-tier allowance of shared-CPU micro-VMs (Firecracker) with persistent volumes and automatic HTTPS on a `*.fly.dev` subdomain.

Listing 8 shows the multi-stage `Dockerfile`: the `builder` stage (Alpine) downloads the official PocketBase v0.23.12 release binary for `linux_amd64` and unzips it; the final stage copies only the binary plus `pb_migrations/` and `pb_hooks/` into a minimal Alpine image, exposes port 8090, and declares `/pb/pb_data` as a volume mount point for the SQLite database and uploaded files. The resulting image is only **18 MB**.

```

1 # --- Stage 1: fetch the PocketBase binary -----
2 FROM alpine:3.20 AS builder
3 ARG PB_VERSION=0.23.12
4
5 RUN apk add --no-cache unzip curl ca-certificates \
6     && curl -fsSL "https://github.com/pocketbase/pocketbase/releases/download/
7         v${PB_VERSION}/pocketbase-${PB_VERSION}_linux_amd64.zip" -o /tmp/pb.zip \
8     && unzip /tmp/pb.zip -d /pb \
9     && rm /tmp/pb.zip
10
11 # --- Stage 2: minimal runtime image -----
12 FROM alpine:3.20
13 RUN apk add --no-cache ca-certificates
14 WORKDIR /pb
15 COPY --from=builder /pb/pocketbase /pb/pocketbase
16 COPY pb_migrations ./pb_migrations
17 COPY pb_hooks ./pb_hooks
18
19 VOLUME /pb/pb_data
20 EXPOSE 8090
21 CMD ["/pb/pocketbase", "serve", "--http=0.0.0.0:8090"]

```

Listing 8: `pocketbase/Dockerfile` — multi-stage build producing an 18 MB PocketBase image.

Listing 9 shows the generated `fly.toml`: a 1 GB persistent volume `pb_data` is mounted at `/pb/pb_data` (matching the `Dockerfile`'s `VOLUME`), `force_https = true` provides automatic TLS termination, and `min_machines_running = 1` keeps the instance always-on (avoiding cold-start latency and keeping SSE realtime subscriptions connected), at the cost of consuming part of the free-tier shared-CPU allowance.

```

1 app = 'michwar-pb'
2 primary_region = 'cdg'
3

```

```

4 [build]
5
6 [[mounts]]
7   source = 'pb_data'
8   destination = '/pb/pb_data'
9
10 [http_service]
11   internal_port = 8090
12   force_https = true
13   auto_stop_machines = 'off'
14   auto_start_machines = true
15   min_machines_running = 1
16
17 [[vm]]
18   size = 'shared-cpu-1x'
19   memory = '256mb'

```

Listing 9: `pocketbase/fly.toml` — Fly.io application configuration.

Figure 5 summarizes the end-to-end deployment pipeline, from source to a publicly reachable HTTPS endpoint.

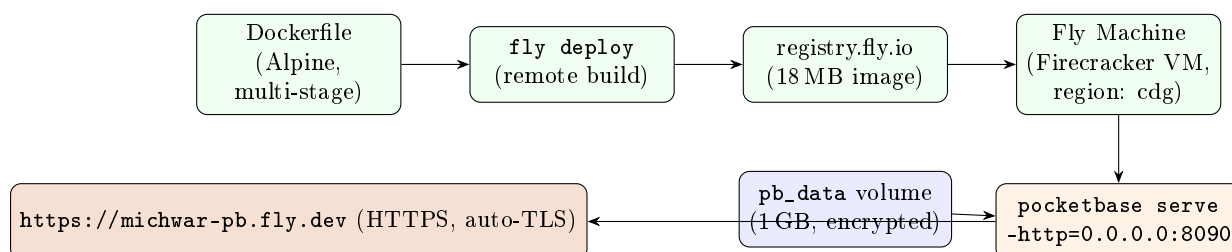


Figure 5: Deployment pipeline: a multi-stage Docker build is sent to Fly.io’s remote builder, pushed to `registry.fly.io`, and run as a Firecracker micro-VM with a persistent volume for `pb_data`, exposed publicly over HTTPS.

3 Results

3.1 Codebase Composition

Table 4 and Figure 6 summarize the size of the resulting codebase by module, measured in lines of code. The Flutter client (`lib/`) totals approximately 8,169 lines across `core/` and the five feature modules, while the PocketBase backend (`pb_hooks/` and `pb_migrations/`) totals 1,374 lines — a useful illustration of how much server-side logic a self-hosted backend requires relative to a “backend-less” Firebase setup, where most of this logic would otherwise live in Cloud Functions written in TypeScript.

3.2 Build Results

The release Android APK built successfully with `flutter build apk -release`, producing `build/app/outputs/flutter-apk/app-release.apk` at approximately **54 MB** (56,613,947 bytes). The build environment was Flutter 3.43.0 (beta channel) with Android SDK 36.1.0 on Windows. The only diagnostics emitted were harmless Java 8 source/target deprecation warnings from the Android Gradle plugin and standard font-asset tree-shaking notices (e.g. `MaterialIcons-Regular.otf` reduced by 99.4%). No application code changes were required beyond the dependency and API-version fixes described in §3.5.

Table 4: Lines of code by module

Module	Lines of code
lib/core/ (models, services, routing, theme, providers)	2,444
lib/features/authentication/	845
lib/features/passenger/	1,768
lib/features/driver/	1,659
lib/features/ride_engine/	717
lib/features/shared/	736
Flutter client total	8,169
pocketbase/pb_hooks/ (business logic)	1,059
pocketbase/pb_migrations/ (schema)	315
PocketBase backend total	1,374

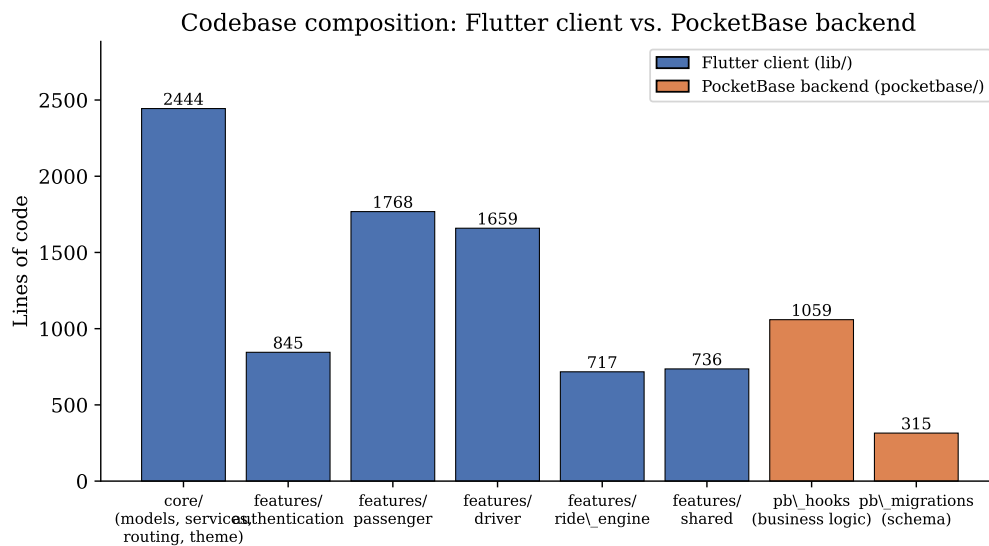


Figure 6: Lines of code per module. The Flutter client (blue) is roughly six times larger than the PocketBase backend (orange), reflecting that the backend logic is concentrated in nine compact hook files plus declarative schema migrations.

3.3 Deployment Outcome

The containerized PocketBase backend was deployed to Fly.io under the application name `michwar-pb` in the `cdg` (Paris) region. The final, successful deploy produced:

- A Docker image of **18 MB** pushed to `registry.fly.io/michwar-pb`.
- A dedicated IPv6 and a shared IPv4 address, with automatic HTTPS via Fly's edge proxy.
- A running Firecracker machine with a 1 GB encrypted persistent volume mounted at `/pb/pb_data`.
- A publicly reachable endpoint at `https://michwar-pb.fly.dev`, with the PocketBase Admin UI at `/_` and the REST/realtime API at `/api/`.

`fly status` confirmed the machine state as `started`, and `fly logs` showed PocketBase applying all nine schema migrations successfully on first boot and beginning to serve HTTP traffic. The release APK was then rebuilt with `-dart-define=POCKETBASE_URL=https://michwar-pb.fly.dev` (§2.7), producing a final distributable artifact that requires no configuration on the end user's device or network.

3.4 Worked Example: Fare and Commission Computation

To validate the pricing engine described in §2.5.1, consider a completed `standard`-tier ride of 8.4 km over 22 minutes, completed by a Tier-1 (Base) driver:

$$\text{raw} = 100 + (8.4 \times 35) + (22 \times 5) = 100 + 294 + 110 = 504 \quad (1)$$

$$\text{tiered} = 504 \times 1.0 = 504 \quad (2)$$

$$\text{baseFare} = \max(150, \text{round}_5(504)) = 505 \quad (3)$$

$$\text{surcharge} = 3 \quad (\text{deterministic}, \in [1, 4]) \quad (4)$$

$$\text{commission} = \text{round}_1(505 \times 0.15) = 76 \quad (5)$$

$$\text{driverPayout} = 505 - 76 = 429 \quad (6)$$

$$\text{companyRevenue} = 76 + 3 = 79 \quad (7)$$

$$\text{totalFare} = 505 + 3 = 508 \quad (8)$$

$$\text{pointsAwarded} = \lfloor 508/100 \rfloor \times 2 = 10 \quad (9)$$

Figure 7 visualizes the resulting split of the 508 DZD total fare. This is the exact computation performed inside `POST /api/michwar/rides/:id/complete`; the breakdown is persisted both on `rides.fare` and as a `wallet_transactions` ledger row.

3.5 Issues Encountered and Resolutions

The path from a migrated source tree to a live, publicly reachable backend surfaced a sequence of distinct issues spanning the Flutter toolchain, the Windows shell, a backend SDK version mismatch, and the PocketBase server itself. Figure 8 and Table 5 summarize them chronologically; each is discussed in turn below.

The final issue is the most architecturally significant: it was *not* a transient network problem despite occurring in the middle of a sequence of genuine network issues, and it would have affected *any* deployment target (Fly.io, a VPS, a local Docker run against an empty volume), since it is intrinsic to running the existing migrations against a PocketBase 0.23+ binary on a fresh database. Diagnosing it required separating "flyctl cannot report status back to the Fly API" (a local network problem, visible as DNS lookup failures for `api.fly.io` and `api.depot.dev`) from "the container that Fly *did* successfully start immediately exited" (an application-level problem, visible only via `fly logs`, which showed `pocketbase` printing the migration error and the init process exiting with code 0 — a "successful" exit from PocketBase's perspective after a fatal

Fare breakdown -- example completed ride
(8.4 km, 22 min, standard tier, Tier-1 driver)
Total fare paid: 508 DZD

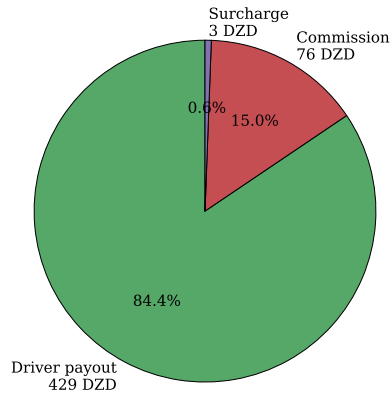


Figure 7: Worked example: fare breakdown for an 8.4 km / 22 min standard-tier ride completed by a Tier-1 driver. The driver receives 429 DZD (84.4%), the company receives 79 DZD in commission + surcharge (15.6%), for a total fare of 508 DZD.

Build & deployment troubleshooting timeline

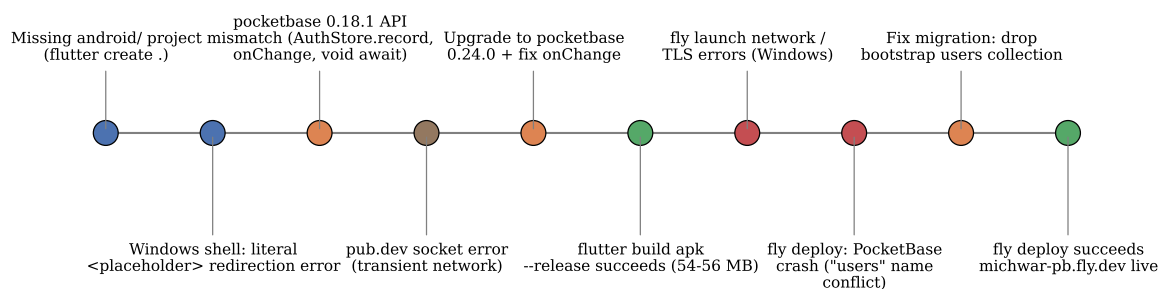


Figure 8: Chronological timeline of build and deployment issues, from the initial `flutter build apk` attempt to the live Fly.io deployment.

Table 5: Build and deployment issues, causes, and resolutions

Symptom	Root cause	Resolution
<code>flutter build apk</code> : "The system cannot find the file specified"	The project was authored without ever running <code>flutter create</code> , so <code>android/</code> (and other platform folders) did not exist	Ran <code>flutter create .</code> from the project root, which scaffolds platform projects without touching <code>lib/</code>
Same error persisted after scaffolding, with <code>-dart-define=POCKETBASE_URL=</code>	On Windows <code>cmd</code> , the literal <code></></code> placeholder characters in the example <code>POCKETBASE_URL=<inter></code> interpreted as I/O redirection operators	Substituted a real IP/URL with no angle brackets, e.g. <code>http://192.168.1.50:8090</code>
Dart compile errors: <code>AuthStore.record</code> not defined, <code>onChange</code> not invocable, <code>void</code> awaited	<code>pocketbase</code> Dart SDK pinned at 0.18.1, whose <code>AuthStore</code> API (model, callback-based <code>onChange</code>) differs from the 0.24.x API used by the ported code	Bumped <code>pocketbase</code> : <code>^0.18.1</code> $\rightarrow$ <code>^0.24.0</code> ; rewrote <code>signOut</code> as <code>Future<void></code> ; rewrote <code>authStateChanges()</code> to consume <code>AuthStore.onChange</code> as a <code>Stream<AuthStoreEvent></code> (Listing 6)
<code>flutter pub get</code> : "Got socket error trying to find package pocketbase at <code>https://pub.dev</code> "	Transient network failure during dependency resolution	Retried <code>flutter pub get</code> ; succeeded on the second attempt
<code>fly launch</code> / <code>fly deploy</code> : repeated <code>read tcp ... wsarecv: A connection attempt failed, dial tcp ... connectex: A connection attempt failed, lookup api.fly.io: no such host</code>	Intermittent connectivity from the Windows development machine to Fly's API (consistent with TLS-inspecting antivirus/firewall or unstable Wi-Fi/hotspot)	Diagnosed via <code>Test-NetConnection api.fly.io -Port 443</code> (raw TCP succeeded while HTTPS API calls failed); retried after switching network and answering launch prompts with the documented values (<code>n</code> to "tweak settings", <code>y</code> to "use this fly.toml")
<code>fly deploy</code> : image built (18 MB) and IPs provisioned, but "Error: timeout reached waiting for machine's state to change"	<code>fly logs</code> revealed the container itself crashed on boot: Failed to apply migration <code>1700000001_users.js</code> : <code>name: Collection name must be unique (case insensitive)</code>	<code>PocketBase</code> ≥ 0.23 auto-creates a default <code>users</code> auth collection on a fresh database; the migration now deletes this bootstrap collection before creating the application's <code>users</code> collection (Listing 1)

startup error). `fly status` additionally showed the machine state as `stopped`, confirming the container was not crash-looping but had cleanly exited once.

4 Discussion

4.1 Architectural Trade-offs: Self-Hosting versus a Managed BaaS

Migrating from Firebase to PocketBase trades a fully managed, infinitely scalable (but billed-per-use and closed-source) backend for a single self-contained binary that the project fully owns. The practical consequences observed in this project are:

- **Operational simplicity at small scale.** The entire backend — schema, business logic, file storage, and admin UI — is a single 18 MB Docker image plus one SQLite file. Deployment to a free-tier Fly.io machine took a handful of commands and no managed-database provisioning.
- **Explicit, auditable business logic.** Every financially sensitive operation is a ~30–130 line JavaScript function in `pb_hooks/`, each wrapped in `$app.runInTransaction()`. This is arguably *more* auditable than a set of Firestore security rules plus separately deployed Cloud Functions, because the data model, the access rules, and the business logic all live in one repository and one runtime.
- **Scaling is now the operator's responsibility.** A single 256 MB shared-CPU machine and an embedded SQLite database are adequate for development and small-scale piloting, but horizontal scaling (read replicas, connection pooling, multi-region) would require additional engineering that a managed BaaS would otherwise absorb.
- **No vendor billing surprises.** The free Fly.io tier (3 shared-cpu-1x VMs) comfortably hosts `michwar-pb` with `min_machines_running = 1`; there is no per-document-read or per-function-invocation billing to monitor.

4.2 Robustness of the Transactional Hook Pattern

A central design decision — carried over faithfully from the original Cloud Functions design — is that *no client write can directly produce a billable number*. The `rides`, `wallets`, `wallet_transactions`, and `drivers` financial fields all have `update/delete` API rules of `null`, and the only way to mutate them is through one of the seven `/api/michwar/...` hook routes, each gated by `$apis.requireAuth()` and wrapped in `$app.runInTransaction()`. This pattern provides two guarantees that are difficult to achieve with purely declarative security rules: (1) *atomicity* across multiple collections (a completed ride atomically updates `rides`, `drivers`, `wallets`, `wallet_transactions`, and `users` in one transaction), and (2) *race-safety* for contended operations (the `/accept` route re-checks `ride.driver` is still empty *inside* the transaction, so two drivers tapping "Accept" simultaneously cannot both succeed).

4.3 Lessons Learned

Several lessons emerged that generalize beyond this specific project:

1. **Pin and verify SDK versions against the server version they target.** The `pocketbase` Dart SDK's `AuthStore` API changed its shape (`callback` → `stream`, `model` → `record`) between 0.18.x and 0.24.x. Code written against one version fails to *compile* (not just run) against the other, so a version bump must be treated as a small porting task, not a routine dependency update.
2. **A framework's bootstrap behaviour can silently change.** PocketBase 0.23 began auto-provisioning a default `users` auth collection on first boot. Migrations written against an earlier assumption ("the database starts empty") must be defensive — e.g. check for and remove a colliding bootstrap collection before creating the application's own.

3. **Separate "my tooling can't report status" from "my application failed."** The Fly.io deployment exhibited both a genuinely flaky local network connection *and* a real application crash in the same deploy attempt. `fly status` (machine state) and `fly logs` (container stdout/stderr) were the tools that disambiguated the two; relying on `flyctl`'s own error message alone would have led to chasing the network issue indefinitely.
4. **Windows shell quoting is a recurring trap for copy-pasted commands.** Placeholder syntax such as `<your-ip>` is idiomatic in documentation but is interpreted as redirection by `cmd.exe`/PowerShell, producing a misleading "file not found" error that has nothing to do with files.
5. **`-dart-define` is an effective mechanism for environment-specific builds without source forks.** The same `lib/core/services/pocketbase_service.dart` serves the emulator, LAN testing, and the public production URL, selected entirely at build time.

4.4 Limitations

The system as deployed has several known limitations, documented in `DEPLOYMENT.md`'s verification checklist:

- Push notifications (FCM) were intentionally dropped in favour of PocketBase realtime (SSE) subscriptions; `users.fcmTokens` remains an unused placeholder field for a future push provider.
- `POST /api/michwar/wallet/topup` currently credits a driver's wallet directly on call, which is appropriate for testing but must be placed behind a verified payment-provider webhook (e.g. CIB/EDAHABIA) before handling real money.
- The single 256 MB Fly.io machine and embedded SQLite database are not designed for high-concurrency production traffic; horizontal scaling would require migrating to a server-based database (e.g. PostgreSQL, which PocketBase 0.23+ supports as an alternative to SQLite) and multiple machines behind Fly's load balancer.
- The live-trip-sharing feature (`live_shares` collection) issues a public URL but assumes a separately hosted static tracking page; none was built as part of this work.
- Legacy Firebase artifacts (`.firebaserc`, `firebase.json`, `firestore.rules`, `functions/`, etc.) remain in the repository as dead files pending manual removal.

4.5 Future Work

Natural extensions of this work include: (1) wiring a real payment provider's webhook into `/api/michwar/wallet/topup`; (2) building the live-trip-sharing static page described in §2.5; (3) adding automated integration tests against a disposable PocketBase instance (e.g. in CI, spinning up the Docker image and exercising the `/api/michwar/...` routes); (4) evaluating PocketBase's PostgreSQL backend for higher-concurrency production use; and (5) building and publishing iOS/web release artifacts alongside the Android APK, using the same `-dart-define=POCKETBASE_URL` mechanism.

5 Conclusion

This report has documented the complete migration of the MICHWAR ride-hailing application's backend from Google Firebase to a self-hosted PocketBase instance, and the subsequent build and public deployment of the resulting system. The migration preserved the original architecture's central guarantee — that all financially sensitive state transitions are computed and persisted atomically, server-side, inside transactions — by re-expressing the nine Firestore collections as PocketBase collections with locked-down API rules, and the eight Cloud Functions as `$app.runInTransaction()` hook routes totalling roughly 1,400 lines of JavaScript and migration code. The Flutter client (8,169 lines) required only targeted changes: a new PocketBase

service/auth layer, updated Riverpod providers for SSE-based realtime subscriptions, and a dependency bump to align with the PocketBase Dart SDK's 0.24.x `AuthStore` API. The resulting Android release APK (≈ 54 MB) was successfully built and, after diagnosing and fixing a PocketBase bootstrap-collection collision that initially crashed the deployed container, the backend is now live at <https://michwar-pb.fly.dev> on Fly.io's free tier — giving MICHWAR a fully self-owned, zero-recurring-cost backend that any downloaded APK can reach from any network, with no manual configuration required by the end user.

References

1. PocketBase — *Open Source backend in 1 file*. <https://pocketbase.io/docs/>
2. Flutter — *Build apps for any screen*. <https://docs.flutter.dev/>
3. Riverpod — *A reactive caching and data-binding framework*. <https://riverpod.dev/>
4. `go_router` — declarative routing package for Flutter. https://pub.dev/packages/go_router
5. Fly.io — *Application platform for full stack apps*. <https://fly.io/docs/>
6. pocketbase Dart SDK (v0.24.x). <https://pub.dev/packages/pocketbase>
7. Niemeyer, G. *Geohash*. <https://en.wikipedia.org/wiki/Geohash>